MERHABA ICT SOLUTION

Software Engineering Capability Assessment

FULL-STACK DEVELOPER EVALUATION PROJECT

5-Day Take-Home Technical Challenge

Candidate	Khalid	**Role Assessed**	Full-Stack Developer
Duration	5 Working Days	**Document Date**	19 August 2026
AI Usage	Fully Permitted	**Company**	Merhaba ICT Solution

1. PURPOSE OF THIS EVALUATION

Merhaba ICT Solution is building modern software products for the hospitality industry (restaurants, hotels, and related services). We are evaluating Khalid for a Full-Stack Developer position on our engineering team.

This 5-day project is designed to assess real-world software engineering capability — not just coding speed. We care about architectural judgment, code quality, product thinking, and the ability to deliver a coherent, maintainable system under realistic constraints.

Important: You are explicitly allowed and encouraged to use AI tools (ChatGPT, Claude, Cursor, GitHub Copilot, etc.) to generate code, design schemas, write tests, and draft documentation. We evaluate the final system quality, your understanding of the solution, and the engineering decisions you make — not whether you wrote every line by hand.

2. PROJECT OVERVIEW

Project Name: Merhaba Order Desk — Restaurant Operations MVP

Build a functional fullstack web application that helps a small-to-medium restaurant manage its daily operations: menus, tables, and customer orders.

This is a scoped MVP. Focus on correctness, clarity, and solid engineering foundations rather than trying to build every possible feature.

3. FUNCTIONAL REQUIREMENTS (MUST HAVE)

3.1 Authentication & Authorization

- Support two roles: **Admin** and **Staff** (waiter/cashier).
- Registration and login with email + password (or equivalent).
- Secure token-based authentication (JWT recommended) or secure session handling.
- Protect routes/APIs according to role (Admin can manage menu & users; Staff can manage orders & tables).

3.2 Menu Management

- Categories (e.g., Starters, Main Course, Drinks, Desserts).
- Menu items with: name, description, price, category, availability flag, optional image URL.

- Full CRUD for categories and items (Admin only).
- Staff can view available menu items when creating orders.

3.3 Table Management

- Represent restaurant tables (table number/name, capacity, current status).
- Statuses: **Available**, **Occupied**, **Reserved**, **Cleaning** (or similar sensible set).
- Ability to change table status and assign/unassign an active order.

3.4 Order Management

- Create a new order linked to a specific table.
- Add / remove / update quantities of menu items on an order.
- Order statuses: **Pending** → **Preparing** → **Ready** → **Served** → **Paid** / **Cancelled**.
- Calculate order total automatically (price × quantity).
- View list of active orders and order history (basic filtering by status or date is a plus).

3.5 Dashboard / Overview

- Simple overview showing: number of occupied tables, active orders by status, and today's order count or revenue (basic).
- Quick navigation to manage tables, menus, and orders.

4. NICE-TO-HAVE (OPTIONAL — IF TIME ALLOWS)

- Real-time order status updates (WebSockets / Server-Sent Events).
- Simple daily sales summary / basic reporting page.
- Soft delete / audit fields (created_at, updated_at, created_by).
- Basic input validation and user-friendly error messages on the frontend.
- Responsive design that works reasonably on tablet (restaurant floor use).
- Seed data script so reviewers can explore the system quickly.

5. TECHNICAL REQUIREMENTS & CONSTRAINTS

You may choose the technology stack that best demonstrates your strengths. We value good decisions over following a prescribed stack.

Recommended (not mandatory) direction:

- Backend: Node.js (Express/Nest/Fastify), Go, Python (FastAPI/Django), or similar mature option.
- Frontend: React, Next.js, Vue, or Svelte — modern component-based UI.
- Database: PostgreSQL strongly preferred (or MySQL / SQLite for simplicity).
- API style: Clean RESTful JSON API (GraphQL is acceptable if you are strong with it).

Hard Requirements:

- Clear separation between frontend and backend (even if monorepo).
- Proper environment configuration (.env or equivalent) — no secrets committed.
- Database migrations or schema definition that can be applied cleanly.
- Meaningful error handling and basic input validation on both API and UI.
- The application must be runnable locally with clear instructions.

6. RULES & REQUIREMENTS YOU MUST FOLLOW

These are non-negotiable expectations. Following them is part of the evaluation.

1. AI is allowed — ownership is required

You may generate large portions of the code with AI. However, you must fully understand the architecture, data model, key algorithms, and trade-offs. Be prepared to explain and defend every significant decision in a follow-up technical discussion.

2. Git repository is mandatory

All work must live in a Git repository (GitHub, GitLab, or Bitbucket). Provide access to the evaluation team. Commit history should show meaningful, logical commits — not one giant dump at the end.

3. Professional README is required

The repository must contain a clear README.md that includes: project overview, tech stack, setup steps, how to run (backend + frontend + database), default credentials (if any), and known limitations.

4. Clean code & structure

Prefer readable, maintainable code over clever one-liners. Use consistent naming, reasonable folder structure, and avoid unnecessary complexity. Comments should explain 'why', not 'what'.

5. Security basics

Never hard-code secrets. Hash passwords properly. Protect authenticated routes. Sanitize inputs. Do not leave debug endpoints or wide-open CORS in the final submission.

6. No proprietary code

Do not copy code from previous employers or closed-source projects. Using open-source libraries and AI-generated code is fine; copying confidential code is not.

7. Honesty about scope

If you cannot finish a feature, document what is incomplete and why. A working core with clear notes is far better than a broken 'complete' system.

8. Time discipline

This is a 5-day evaluation. Do not spend more than the allocated time. We are assessing judgment under realistic constraints, not how many nights you can stay up.

7. SUGGESTED 5-DAY EXECUTION PLAN

This is a recommended schedule only. Adapt it to your working style.

Day	Focus	Expected Outcomes
Day 1	Foundation	Project setup, database schema design, authentication (register/login + roles), basic project structure & README skeleton.
Day 2	Core Domain	Menu categories & items CRUD (API + UI). Table management (CRUD + status). Seed data.
Day 3	Orders	Order creation, line items, status transitions, totals calculation, linking orders to tables.
Day 4	Dashboard & Polish	Overview dashboard, UI/UX improvements, error handling, basic validation, responsive checks.
Day 5	Hardening & Docs	Final testing, documentation polish, deployment notes or demo video, known issues list, submission package.

8. REQUIRED DELIVERABLES

At the end of the 5 days, please submit the following:

1. **Git repository link** with full source code and meaningful commit history.

2. **Complete README.md** covering setup, running instructions, tech choices, and known limitations.
3. **Short architecture / decision notes** (can be part of README or a separate DECISIONS.md) explaining key choices.
4. **Working local demo** — reviewers must be able to run the system following your instructions.
5. **Optional but valued:** Deployed demo link (Vercel, Railway, Render, etc.) or a short Loom/screen recording walkthrough.
6. **Credentials** for any seeded admin and staff accounts.

9. HOW WE WILL EVALUATE YOUR WORK

Area	What We Look For	Weight
Architecture & Design	Clear separation of concerns, sensible data model, thoughtful API design, scalability awareness.	25%
Code Quality	Readability, consistency, naming, error handling, absence of obvious anti-patterns.	20%
Functional Completeness	Core features work end-to-end. Edge cases handled reasonably. Status flows make sense.	20%
Product Thinking	UI is usable, flows feel natural for restaurant staff, priorities are sensible.	15%
Documentation & Process	README quality, commit hygiene, ability to explain decisions, honesty about trade-offs.	10%
Engineering Judgment	What you chose to build vs. skip, how you used AI, time management, security basics.	10%

After reviewing the submission we will schedule a technical conversation. In that discussion we will explore architecture decisions, trade-offs, how AI was used, and how you would evolve the system in a real production environment.

10. SUBMISSION INSTRUCTIONS

Please send the following to the hiring contact at Merhaba ICT Solution before the deadline:

- Repository URL (with access granted if private)
- Brief message confirming completion and highlighting anything you want us to notice
- Demo credentials and any deployment / video link

We look forward to reviewing your work and learning how you approach real software problems.

Merhaba ICT Solution

Building practical software for the hospitality industry